

ConcurrencyCheck Attribute in Entity Framework Core

In this article, I will discuss **ConcurrencyCheck Data Annotation Attribute in Entity Framework Core (EF Core)** with Examples. Please read our previous article discussing [TimeStamp Attribute in Entity Framework Core](#) with Examples.

Understanding Concurrency in Databases

Concurrency in databases refers to the situation where multiple transactions or operations attempt to access or modify the same data simultaneously. Without proper concurrency control mechanisms, this can lead to:

- **Lost Updates:** One user's changes overwrite another's without detection.
- **Dirty Reads:** A transaction reads data that has been modified but not yet committed by another transaction.
- **Non-Repeatable Reads:** Data retrieved by a transaction is changed by another transaction before the first transaction is completed.
- **Phantom Reads:** New records added by another transaction are visible to a transaction that re-executes a query.

There are two Types of Concurrency Control. They are as follows:

- **Optimistic Concurrency Control:** Assumes that conflicts are rare and checks for conflicts before committing changes.
- **Pessimistic Concurrency Control:** Locks data resources to prevent conflicts, assuming that conflicts are likely.

EF Core primarily employs Optimistic Concurrency Control, which assumes that conflicts are rare and checks for them only when changes are being saved. EF Core uses attributes like TimeStamp and ConcurrencyCheck to manage concurrent data modifications. In our previous article, we discussed using the TimeStamp Attribute to manage the concurrency, and in this article, I will discuss using the ConcurrencyCheck Attribute.

What is the ConcurrencyCheck Attribute in Entity Framework Core?

The ConcurrencyCheck Data Annotation Attribute can be applied to one or more properties (properties with any data type) of an entity in Entity Framework Core, unlike the TimeStamp Attribute, which is applied only once within an entity and is also a property of the Byte array type.

In Entity Framework Core, when we apply the ConcurrencyCheck Attribute to a property or properties of an Entity, the corresponding column or columns in the database table will be used in the optimistic concurrency check. In other words, Entity Framework Core uses those properties in the where clause when performing the Update and Delete operations.

That means the ConcurrencyCheck Attribute in Entity Framework Core is another way to handle Concurrency Issues. A concurrency issue arises when multiple users or transactions attempt to update/delete the same data simultaneously.

Definition of ConcurrencyCheck Attribute in EF Core:

If you go to the definition of ConcurrencyCheck Attribute, you will see the following. As you can see, this class has a parameterless constructor.

```

namespace System.ComponentModel.DataAnnotations
{
    public sealed class ConcurrencyCheckAttribute : Attribute
    {
        public ConcurrencyCheckAttribute();
    }
}

```

Example to Understand ConcurrencyCheck Attribute in EF Core:

Let us understand ConcurrencyCheck Data Annotation Attribute in Entity Framework Core with an example. In EF Core, we can apply the ConcurrencyCheck Attribute with one or more properties of an Entity, and the properties can be of any data type.

Please create a class file named **Student.cs** within the **Entities** folder and then copy and paste the following code. Here, you can see that we have applied the ConcurrencyCheck Attribute on the Name and RegdNumber Properties.

```

using System.ComponentModel.DataAnnotations;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Student
    {
        public int StudentId { get; set; }
        [ConcurrencyCheck]
        public int RegdNumber { get; set; }
        [ConcurrencyCheck]
        public string? Name { get; set; }
        public string? Branch { get; set; }
    }
}

```

Modifying the Context Object:

Next, modify the context class as follows:

```

using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // To Display the Generated the Database Script
            optionsBuilder.LogTo(Console.WriteLine, LogLevel.Information);
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=StudentDB;Trusted_Connection=True;TrustServ
        }
    }
}

```

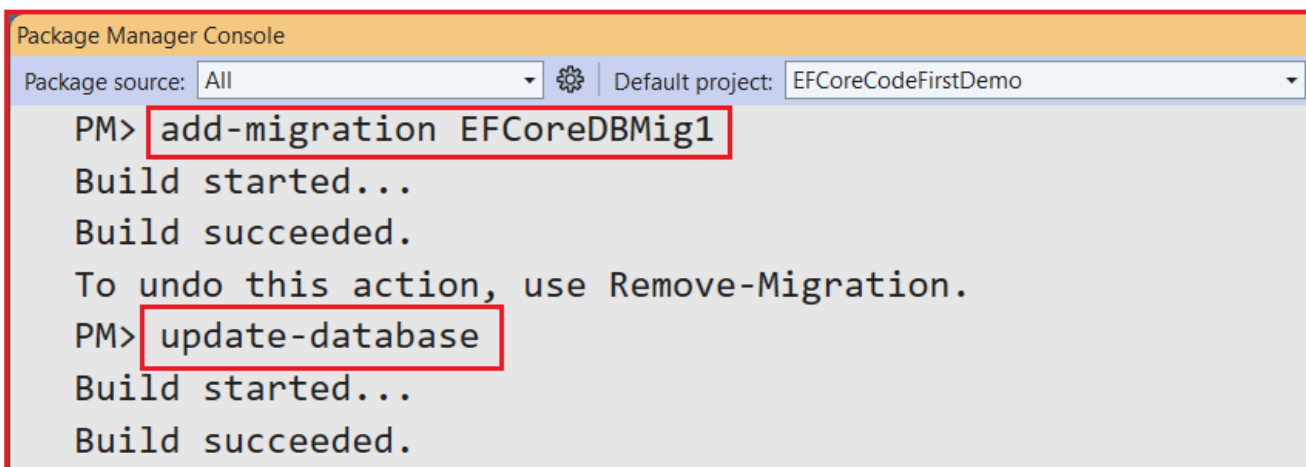
```

//Overriding the OnModelCreating method to add seed data
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Seeding Student data
    modelBuilder.Entity<Student>().HasData(
        new Student { StudentId = 1, Name = "Pranaya", Branch = "(
1001 },
        new Student { StudentId = 2, Name = "Hina", Branch = "CSE'
    },
        new Student { StudentId = 3, Name = "Rakesh", Branch = "CS
    }
    );
}
public DbSet<Student> Students { get; set; }
}
}

```

Generating and Applying Migration:

Now, with the above changes in place, open Package Manager Console and Execute the following **Add-Migration** and **Update-Database** commands as follows.

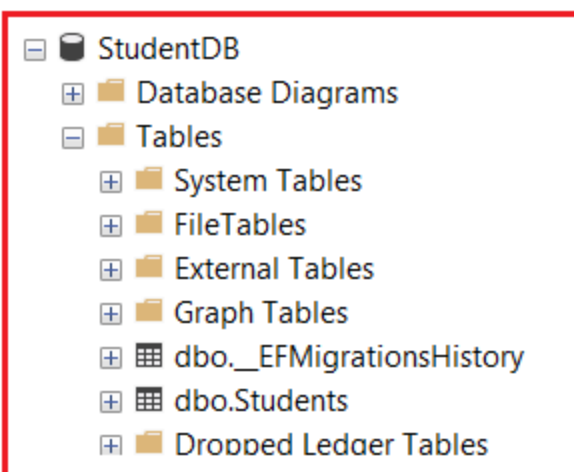


```

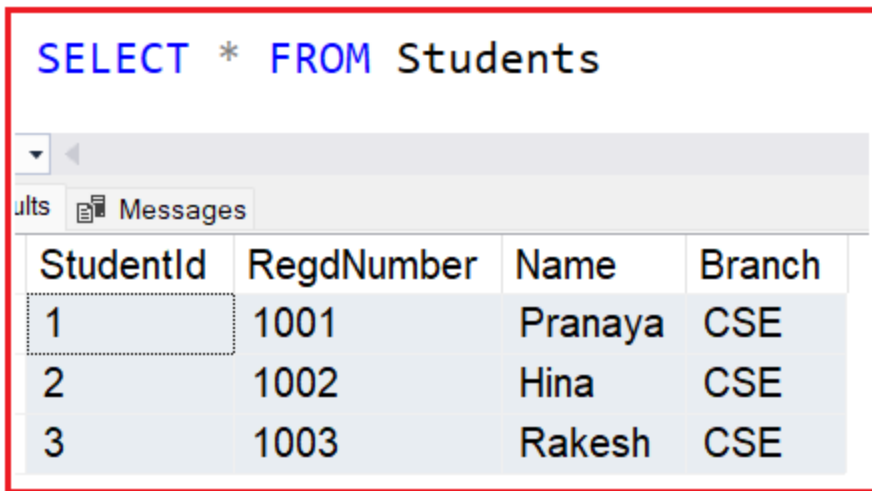
Package Manager Console
Package source: All Default project: EFCoreCodeFirstDemo
PM> add-migration EFCoreDBMig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.

```

Once the above commands are executed, the following StudentDB should have been created with the Students database table.



Now, verify the Students database table. You should see the following seed data for testing purposes.



The screenshot shows a SQL query window with the query `SELECT * FROM Students`. Below the query, there is a table with 4 columns: StudentId, RegdNumber, Name, and Branch. The table contains 3 rows of data. The first row has StudentId 1, RegdNumber 1001, Name Pranaya, and Branch CSE. The second row has StudentId 2, RegdNumber 1002, Name Hina, and Branch CSE. The third row has StudentId 3, RegdNumber 1003, Name Rakesh, and Branch CSE. The table is highlighted with a red border.

StudentId	RegdNumber	Name	Branch
1	1001	Pranaya	CSE
2	1002	Hina	CSE
3	1003	Rakesh	CSE

Handling Concurrency Problem in EF Core:

The ConcurrencyCheck column(s) will be included in the where clause whenever EF Core updates or deletes an entity and calls the SaveChanges method. We have already added a few students with StudentId 1, 2, and 3. Let us update the Name and Branch of the Student whose ID is 1. To do so, modify the Program class as follows.

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    internal class Program
    {
        static void Main(string[] args)
        {
            try
            {
                using EFCoreDbContext context = new EFCoreDbContext();
                //Fetch the Student Details whose Id is 1
                var studentId1 = context.Students.Find(1);
                if (studentId1 != null)
                {
                    studentId1.Name = "Name Updated";
                    studentId1.Branch = "Branch Updated";
                    context.SaveChanges();
                }
                Console.ReadKey();
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Error: {ex.Message}"); ;
            }
        }
    }
}
```

Now, run the above example. You should get the following output. Here, you can see that the where clause uses the Name and RegdNumber columns along with the Primary Key Column to handle the concurrency issues.

```
info: 13-08-2023 10:22:18.061 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (50ms) [Parameters=[@__p_0=?' (DbType = Int32)], CommandType='Text', CommandTimeout='30'
    ]
      SELECT TOP(1) [s].[StudentId], [s].[Branch], [s].[Name], [s].[RegdNumber]
      FROM [Students] AS [s]
      WHERE [s].[StudentId] = @__p_0
info: 13-08-2023 10:22:18.188 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (15ms) [Parameters=[@p2=?' (DbType = Int32), @p0=?' (Size = 4000), @p1=?' (Size = 4000), @p3=?' (Size = 4000), @p4=?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
      SET IMPLICIT_TRANSACTIONS OFF;
      SET NOCOUNT ON;
      UPDATE [Students] SET [Branch] = @p0, [Name] = @p1
      OUTPUT 1
      WHERE [StudentId] = @p2 AND [Name] = @p3 AND [RegdNumber] = @p4;
```

Handling Concurrency Issues using Concurrency Check in Entity Framework Core:

Let us understand how to handle concurrency issues with Entity Framework Core using the ConcurrencyCheck column. Please modify the Program class as follows. Here, we are updating the same student using two different threads simultaneously. Both Method1 and Method2 read the student entity whose ID is 1 and also read the same Name and RegdNumber values, which will be used in the where clause when updating and deleting the records.

Let us assume Method 1 starts updating first. So, he will update the data in the database and also the Name and RegdNumber column values. Now, Method2 tries to update the same entity. If you remember, while updating, it will use the Name and RegdNumber columns in the where clause, but Method1 has already modified the Name and RegdNumber column value. So, Method2 has Name and RegdNumber values that no longer exist in the database, and hence, the Method2 SaveChanges method will throw an exception showing concurrency issues. It might be possible that thread2 starts its execution first; in that case, Method1 will throw an exception.

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    internal class Program
    {
        static void Main(string[] args)
        {
            try
            {
                Console.WriteLine("Main Method Started");
                Thread t1 = new Thread(Method1);
                Thread t2 = new Thread(Method2);
                t1.Start();
                t2.Start();
                t1.Join();
                t2.Join();
                Console.WriteLine("Main Method Completed");
                Console.ReadKey();
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Error: {ex.Message}"); ;
            }
        }
    }
}
```

```

    }
    public static void Method1()
    {
        using EFCoreDbContext context = new EFCoreDbContext();
        //Fetch the Student Details whose Id is 1
        var studentId1 = context.Students.Find(1);
        //Before Updating Delay the Thread by 2 Seconds
        Thread.Sleep(TimeSpan.FromSeconds(2));
        if (studentId1 != null)
        {
            studentId1.Name = studentId1.Name + "Method1";
            studentId1.Branch = studentId1.Branch + "Method1";
            context.SaveChanges();
            Console.WriteLine("Student Updated by Method1");
        }
    }
    public static void Method2()
    {
        using EFCoreDbContext context = new EFCoreDbContext();
        //Fetch the Student Details whose Id is 1
        var studentId1 = context.Students.Find(1);
        //Before Updating Delay the Thread by 2 Seconds
        Thread.Sleep(TimeSpan.FromSeconds(2));
        if (studentId1 != null)
        {
            studentId1.Name = studentId1.Name + " Method2";
            studentId1.Branch = studentId1.Branch + " Method2";
            context.SaveChanges();
            Console.WriteLine("Student Updated by Method2");
        }
    }
}
}
}

```

When you run the above application, you will get the following exception, which makes sense.

```

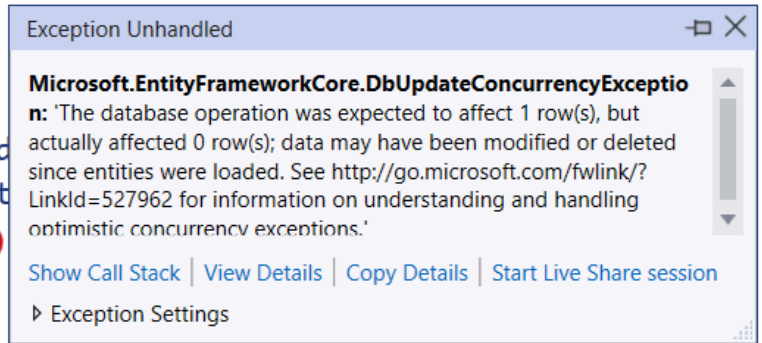
public static void Method1()
{
    using EFCoreDbContext context = new EFCoreDbContext();

    //Fetch the Student Details whose Id is 1
    var studentId1 = context.Students.Find(1);

    //Before Updating Delay the Thread by 2 Seconds
    Thread.Sleep(TimeSpan.FromSeconds(2));

    if (studentId1 != null)
    {
        studentId1.Name = studentId1.Name;
        studentId1.Branch = studentId1.Branch;
        context.SaveChanges();
        Console.WriteLine("Student Updated");
    }
}

```



When Should We Use the ConcurrencyCheck Attribute in EF Core?

Use the ConcurrencyCheck Attribute When:

- **You Have Specific Properties Critical for Concurrency:** If certain properties in your entity are Critical for maintaining data integrity (e.g., Balance in a banking application), marking them with [ConcurrencyCheck] ensures that changes to these properties are tracked for concurrency conflicts.
- **Your Database Does Not Support RowVersion or Similar Features:** In scenarios where the underlying database doesn't support automatic row versioning (like SQL Server's rowversion), [ConcurrencyCheck] provides a manual way to implement concurrency control.
- **You Want to Use Existing Properties Without Adding New Ones:** If you prefer not to introduce additional properties (like a RowVersion byte array), [ConcurrencyCheck] enables you to utilize existing properties for concurrency tracking.

Differences Between ConcurrencyCheck and TimeStamp in EF Core

Please have a look at the following image to understand the Key Differences Between ConcurrencyCheck and TimeStamp in EF Core:

Feature	ConcurrencyCheck	TimeStamp
Scope	Property-level concurrency control	Row-level concurrency control
Concurrency Control	Protects only specific fields from concurrent updates	Protects the entire row from concurrent updates
Data Type	Can be applied to any type of property	Must be applied to a <code>byte[]</code> property (typically called <code>RowVersion</code>)
Use Case	When only particular fields are critical to concurrency	When the entire row of data must be versioned and protected
Automatic Handling	Requires manual marking of each relevant property	Automatically managed by EF Core with versioning
Complexity	Easier for simple scenarios involving specific fields	More robust for complex scenarios where row-level protection is needed
Conflict Detection	Checks if the specific fields marked have changed	Checks if the entire row has changed based on the timestamp/versioning

Performance Considerations Between ConcurrencyCheck and TimeStamp in EF Core

ConcurrencyCheck:

- **Performance:** It is fast for small updates as it only checks changes on specific properties. However, if you mark multiple fields with ConcurrencyCheck, there could be increased performance overhead.
- **Use Case:** It is ideal for systems where only a few critical fields need protection (e.g., updating a bank account's balance).

TimeStamp:

- **Performance:** This can be more performant in row-level concurrency checks because the versioning mechanism is automatic and doesn't involve comparing multiple fields. However, there is slight storage overhead due to the additional `byte[]` column.
- **Use Case:** This is ideal for complex entities where multiple fields can be updated together, and row-level concurrency is needed.

So, in summary, use ConcurrencyCheck for property-specific concurrency control and TimeStamp for row-level concurrency control.

In the next article, I will discuss [Fluent API in Entity Framework Core](#) with Examples. In this article, I explain the **ConcurrencyCheck Data Annotation Attribute in Entity Framework Core** with Examples. I hope you enjoyed this ConcurrencyCheck Attribute in EF Core with Examples article.